PUC Minas

PONTIFÍCIA UNIVERSIDADE CATÓLICA DE MINAS GERAIS

Lista 04 - Ordenação Parte 1

Professora: *Marta Noronha*

Disciplina: *Algoritmos e Estrutura de Dados II*
Data de entrega: *disponível no Verde*

Requisitos

1. Todos os programas deverão ser desenvolvidos na linguagem de programação Java.
2. Cópias, se existirem, serão encaminhadas ao colegiado de coordenação didática do curso.
3. Fique atento ao charset dos arquivos de entrada e saída. Recomenda-se a utilização dos métodos da classe MyIO.java para leitura de dados do teclado. É necessário definir o charset a ser utilizado antes de começar a leitura de dados do teclado, da seguinte forma: `MyIO.setCharset("ISO-8859-1")`.
4. As saídas esperadas, cadastradas no **VERDE** pelo professor, foram geradas empregando-se: `System.out.println()`.
5. Os arquivos *pub.in* e *pub.out* estarão disponíveis no **VERDE** se forem necessários para a execução dessa atividade.
6. Em cada submissão, enviar apenas um arquivo (.java). A regra será necessária para a submissão de exercícios no **VERDE** e no identificador de plágios utilizado na disciplina.
7. A resolução (código) de cada exercício deverá ser submetida ao **VERDE**.
8. A execução do código submetido será realizada automaticamente pelo **VERDE**, mas o código será analisado e validado pelo professor.

Criação da classe Jogo

O conjunto de dados utilizado neste exercício foi disponibilizado em Kaggle.
Implemente uma classe Jogo com os seguintes atributos declarados com modo de acesso privado:

- `rank` (int): classificação (posição) do jogo em relação aos demais.
- `nome do jogo` (String): o nome do jogo.
- `plataforma` (String): Onde o jogo pode ser executado (Wii, DS, entre outros).

- `ano` (int): ano de lançamento.
- `gênero` (String): esportes, corrida, entre outros.
- `editora` (String): empresa que disponibiliza o jogo (produtora).
- `NA_Vendas` (double): Vendas do jogo na América do Norte.
- `EU_Vendas` (double): Vendas do jogo na Europa.
- `JP_Vendas` (double): Vendas do jogo no Japão.
- `Outras_Vendas` (double): Vendas do jogo em outros lugares.
- `Vendas_Global` (double): Vendas global do jogo.

A classe também deve conter, obrigatoriamente, ao menos, dois construtores (*1. padrão (default); 2. (nome do jogo)/plataforma/ano*), e os métodos *gets*, *sets*, métodos `clone()`, `ler()`, `imprimir()` e `toString()`.

O método `clone()` deve retornar um objeto da mesma classe e contendo os mesmos valores de atributos do atual objeto analisado (**vide exemplo do método clone no slide "Unidade 0 - Nivelamento - Ponteiros (C/C++) e Referências (Java)**"postado na disciplina teórica). **Não é permitido usar a interface Cloneable para esta finalidade.**

O método `ler()` deve receber cada linha do arquivo como ***parâmetro***. Cada linha possui os dados de atributos de cada objeto do tipo Jogo. O método deve armazenar os valores contidos em cada linha nos atributos de cada objeto que foi instanciado.

O método `toString()` deve ser implementado para permitir a impressão de um objeto da classe Jogo, sem necessidade de invocação do método `imprimir()`. Isso é feito pela sobrescrita do método `toString()`, um método implementado na super classe **Object**. Esse método retorna, por padrão, uma *String* contendo as informações do objeto, o qual deve ser implementado para apresentar as informações conforme o modelo indicado no fim deste documento. **Clique AQUI para ver o modelo**.

> Em Java, a classe **Object** define os comportamentos mínimos que todos os objetos possuem. Um desses métodos é o **toString()**, cujo propósito é fornecer uma representação textual do objeto. Por padrão, a implementação de toString() na classe Object **retorna uma string** contendo o nome da classe seguido de um código hash, mas é comum que os desenvolvedores sobrescrevam esse método para fornecer uma representação mais significativa, geralmente exibindo os valores dos atributos do objeto.
>
> Todas as classes em Java, inclusive a que contém o método main, herdam da classe Object. Dessa forma, ao sobrescrever o método toString(), o desenvolvedor redefine a forma como o objeto é convertido em string, permitindo uma representação customizada e mais informativa.

O método `imprimir()` é um método público sem retorno, implementado na classe Jogo, que imprime na saída padrão a representação textual de um objeto do tipo Jogo. Para isso, o método deve simplesmente chamar o método de impressão informando, como parâmetro, o objeto por meio da palavra-chave *this*.

```
public void imprimir() {
    System.out.println(this);
}
```

Aqui, `println(Object)` recebe o próprio objeto (`this`), chama automaticamente o método sobrescrito `toString()` e exibe a string formatada conforme o modelo exigido.

Os métodos que não obedecem a forma como devem ser implementados, conforme descrição acima, serão penalizados ainda que o envio no Verde contabilize 100% de acerto. Isso será feito porque o desenvolvedor deve seguir atentamente as regras de implementação de um método, quando houver, considerando os parâmetros e o tipo de retorno que um método deve possuir quando estes são solicitados. Em caso de dúvidas, consulte a Professora.

Após a criação da classe, deve ser criado um mecanismo para processamento de uma entrada de dados. A entrada de dados é dividida em 2 partes:

- Parte 1: Armazenamento de informações contidos no arquivo jogos.txt em vetor
- Parte 2: Pesquisa de informações armazenadas no vetor criado na parte 1.

Parte 1 (Leitura de Arquivo): Armazenamento de informações contidos no arquivo jogos.txt em vetor

Seu programa deve ler um arquivo-texto chamado **"jogos.txt"** que, no VERDE, localiza-se na pasta **"/tmp"**.

O aluno(a) deve preencher um vetor de objetos da classe Jogo com os dados dos diversos jogos informados no arquivo "jogos.txt".

Use um ArrayList<Jogo> para aumentar dinamicamente o *array* contendo os jogos que serão incluídos. Consulte um guia para compreender o funcionamento do *ArrayList* (Clique aqui) .

Cada uma das linhas seguintes apresenta os dados de um jogo, separados pelo símbolo "(Use "\\|"para fazer o split do vetor). Os dados possuem, em ordem, as seguintes informações:

- `rank` (int);
- `nome do jogo` (String);
- `plataforma` (String);
- `ano` (int);
- `gênero` (String);
- `editora` (String);
- `NA_Vendas` (double);
- `EU_Vendas` (double);
- `JP_Vendas` (double);
- `Outras_Vendas` (double);
- `Vendas_Global` (double);

A última linha do arquivo é vazia.

Parte 2: Pesquisa de informações armazenadas no vetor (ArrayList) criado na parte 1.

Após o processamento da primeira parte da entrada de dados, o programa deve processar a entrada contida no arquivo *pub.in*. Cada linha da segunda parte contém, em ordem, as seguintes informações:

- nome do jogo;
- ano;

- editora;
- plataforma.

A última linha do arquivo *pub.in* contém a palavra FIM.

Os dados de jogos, contidos em cada entrada (linha) lida no arquivo *pub.in*, devem ser pesquisados no vetor de jogos.

Armazene as entradas (dados de jogos) encontradas no vetor de jogos em um ArrayList que, posteriormente, será utilizado como fonte de dados para os algoritmos de ordenação. Em caso de dúvidas sobre como implementar um ArrayList, consulte um material **clicando aqui**.

O programa deve contabilizar a quantidade de jogos lidos na entrada padrão (arquivo *pub.in*), que **foram** encontrados no arquivo *jogos.txt*. A primeira linha do arquivo de saída deverá ser mostrar a informação:

Quantidade de jogos encontrados: {total de jogos contabilizados no *pub.in*}
(Obs: as chaves não devem ser impressas)

Parte 3: Ordenação do vetor armazenado na Parte 2.

Nesta etapa será feita a ordenação do ArrayList utilizando 3 algoritmos de ordenação:

- Bubble sort;
- Selection sort; e,
- Insertion sort.

Para que os métodos utilizem a mesma entrada, devem ser gerados 3 clones do ArrayList criado na Parte 2 (**Não é permitido usar a interface Cloneable para esta finalidade.**). Para atender a essa finalidade, implemente um método na classe Jogo que receba o ArrayList (criado na Parte 2) como parâmetro e retorne o clone desse ArrayList. Dentro do método, todos os objetos deverão ser clonados (por meio da chamada individual de cada objeto ao método clone(), implementado na etapa da criação da classe Jogo) para o ArrayList que será retornado.

Serão criados 3 clones do ArrayList (criado na Parte 2). Cada clone do ArrayList será utilizado como entrada em um dos métodos de ordenação descritos acima.

Parte 3.1: Critérios de ordenação.

A ordenação de elementos no vetor será feita com base em atributos da classe Jogo. Estes atributos devem ser utilizados para comparar se um dado jogo é maior (ou menor) que outro de acordo com os seguintes critérios que são utilizados para ordená-los:

- Nome do jogo (crescente);
- Plataforma (crescente) - só é usado se os nomes forem idênticos;
- Vendas global (decrescente) - só é usado se nome e plataforma forem idênticos.

Considere a Tabela 1 como exemplo de jogos antes de aplicar os critérios de ordenação.

Após aplicar os critérios de ordenação (Nome ↑, Plataforma ↑, Vendas ↓), será obtida a ordenação apresentada na Tabela 2:

Após o processamento de um determinado método de ordenação, devem ser impressas as informações dos jogos ordenados, segundo o critério previamente informado.

Nome	**Plataforma**	**Vendas Globais**
FIFA 21	PS4	0,09
FIFA 21	Xbox One	0,08
Minecraft	PC	0,4
FIFA 21	PS4	0,12

Tabela 1: Lista de jogos antes da ordenação

Nome	**Plataforma**	**Vendas Globais (milhões)**
FIFA 21	PS4	0,12
FIFA 21	PS4	0,09
FIFA 21	Xbox One	0,08
Minecraft	PC	0,4

Tabela 2: Lista de jogos após a ordenação

Para cada jogo, deve ser escrito na saída padrão uma linha contendo as informações do jogo, com os dados do registro correspondente. Além da venda global do jogo, uma função deve ser implementada para informar em que região ou país o jogo foi mais vendido (NA_vendas, EU_Vendas, JP_Vendas ou Outras_Vendas). Caso mais de uma região ou país tenha tido o mesmo número de vendas, **informar somente o primeiro encontrado com a maior venda**. A saída padrão deve obedecer ao seguinte formato (*não incluir chaves*):

[*Nome do jogo*] [*Plataforma*] [*Vendas global*] {*Rank*}. {*Gênero*}. {*Editora*}. Mais vendido:{*País ou região mais vendido*}.

Este formato de saída deve ser implementado utilizando o método `imprimir()` mencionado na Parte 1 deste enunciado. Exemplo de uma possível saída ordenada:

[*Dynasty Warriors: Gundam 2*] [*PSV*] [*0.06*] *12452. Adventure. Aksys Games. Mais vendido: NA_Vendas.*

[*Dynasty Warriors: Gundam 3*] [*PS*] [*0.67*] *3229. Role-Playing. Square. Mais vendido: JP_Vendas.*

[*Dynasty Warriors: Gundam 3*] [*PS*] [*0.63*] *3228. Role-Playing. Square. Mais vendido: JP_Vendas.*

Dicas para melhoria do código (não obrigatórias):

- *Crie métodos para comparação de objetos ("ehMaior" e "ehMenor"), contendo os critérios indicados;*
- *Para o método "ehMenor", inverta a ordem (crescente ou decrescente) para cada atributo. Logo, os critérios para ordenação "ehMenor" serão:*
  - *Nome do jogo (decrescente);*
  - *Plataforma (decrescente) - só é usado se os nomes forem idênticos;*
  - *Vendas global (crescente) - só é usado se nome e plataforma forem idênticos.*
- *Crie uma classe para cada algoritmo de ordenação, instanciando um objeto de cada classe de algoritmo para cada ArrayList que será ordenado. Cada classe de algoritmo deve possuir um método ordenar(...) que recebe o ArrayList como parâmetro e retorna a mesma lista já ordenada. (Opcional: Em vez de parâmetro, você pode passar o ArrayList no construtor, armazená-lo como atributo e retornar sua referência pelo método ordenar());*

Parte 3.2: Resumo da ordenação.

Durante o processo de ordenação, devem ser armazenadas as seguintes métricas:

- Número de comparações entre jogos;
- Número de movimentações (trocas) de posições no vetor.

Ao fim da execução de cada um dos algoritmos de ordenação, deve ser exibido um resumo do número de comparações e movimentações executadas. O resumo deve ser impresso com o seguinte padrão:

```
## {nome_metodo} [COMPARACOES] [{num_comparacoes}] [MOVIMENTACOES] [{num_movimentacoes}]
```

Onde:

- "NOME-METODO"= SELECTION ou BUBBLE ou INSERTION;
- "{num_comparacoes}"= número de comparações divididas por 1000, sem casas decimais;
- "{num_movimentacoes}"= número de movimentações divididas por 1000, sem casas decimais.

Após cada valor do número de comparações e movimentações, acrescentar a letra "k", para indicar que a contagem de números está em função de 1000.

Exemplo de saída:

```
## BUBBLE [COMPARACOES] [7k] [MOVIMENTACOES] [8k]
```

Após a conclusão do trabalho, observe a quantidade de comparações e movimentações realizadas por cada um dos algoritmos. Analise os resultados e avalie se os mesmos estão de acordo com o esperado, a partir do estudo da disciplina teórica.

Dicas (não obrigatórias):

- *Crie uma classe log, para gerenciamento da contagem de operações;*
- *Conte cada um das cláusulas de comparação (IF).*
- *Quando houver IFs aninhados, não se esqueça de contar as cláusulas anteriores, que já foram avaliadas.*
- *A criação de um método único para comparação ("ehMaior" e "ehMenor") facilita o processo de comparação.*